VU AI HUB · LECTURE NOTES

Logic and Sets for AI

Year 1, February to March 2026

Written by Inaaya and shared on Notion. Original page: cheddar-wall-f3b.notion.site/2fa9801a93098038b87ff70fab2cbd9a

Contents

1. 4th of February (Logic)
2. 5th of February (Sets)
3. 12th of February (Sets)
4. 18th of February (Logic)

4th of February (Logic)

Propositional logic

Syntax

Propositions: statements (symbols + operators)

Symbols: things in the universe, ex. $P_{1,2}$

Operators: and, or, not, implies, equivalent, ex. $P_{1,2}$ or ex. $P_{2,1}$

Semantics

Propositions can be True or False, ex $S_{1,2}$ thus $W_{1,3}$ or $W_{2,2}$

Calculations

Next Lecture

Propositional logic syntax

P_1 and P_2 can be used as sentences, by using the next symbols in between them

- Negation ¬
- Conjunction ∧
- Disjunction ∨
- Implication ⇒
- Biconditional ⇔

Propositional logic semantics

A sentence can either be true or false
Complex statements are compositions of simple statements. Their truth value is calculated from the meaning of its parts

- Conjunction ∧ *and*
  - only true when both sides are true
- Disjunction ∨ *or*
  - true when at least one side is true
- Implication ⇒
  - true when both sides are true or both sides are false
  - also true when the right side is true
- Double implication ↔

- only true when both sides are true or both sides are false
- Semantically significant ≡
  - only true when both truth tables have the same values in the same order

---

A knowledge base is a set of sentences that we assume to be true, written in curly bracket notation

Models are interpretation functions of the variables, or validations, which evaluate all formulas in a KB to true

- In other words, truth values that make a knowledge base true

5th of February (Sets)

Set Theory for Computer Science

Sets: Definition and Notation

A set is a structure representing an *unordered* collection of zero or more *different* objects

- set theory deals with operations between, relations among, and statements about sets
- Notation: { }

We use variables, ex. S, T, U etc.

- We denote a set S in curly brackets with all the variables inside: S = {a, b, c}

Set builder notation: for any proposition P(x) (statement), {x/P(x)} is the set of all x such that P(x)

- ex. {x | x is an integer where x > 0 and x <5} = {1, 2, 3, 4}

Sets are unordered so {a, b, c} = {b, c, a} = {a, c, b}

Sets are distinct (unique) so {a, b, c} = {a, a, b, b, b, c}

Sets can be never ending (infinite)

Prototype Notation:

- Describing what's in the set instead of writing everything down

Element and Subset

x ∈ S means "x is an element or member of set S"

- "x is in S"
- 4 ∈ {1, 2, 3, 4}

Set A is equal to set B *iff* they have the same exact members

- x ∉ S *iff* x is not in S
- 5 ∈ {1, 2, 3, 4}

∅, null or empty set, is the unique set that contains no elements

The symbol for subset is ⊂ (proper subset) or ⊆ (subset or equal to)

- S ⊆ T, every element of S is also an element of T, and the other way around
  - {1, 2, 3, 4} = {4, 3, 1, 2} = {4, 3, 3, 2, 1, 2}
- 2, 3 ⊆ {1, 2, 3, 4}
- ∅ is a subset of every set
- S ⊂ T, every element of S is in T, but not the other way around

The symbol for superset is ⊃ (proper superset) or ⊇ (superset or equal to)

- S ⊇ T, every element in T is in S, but not necessarily vice versa

The elements in a set may be sets themselves

Fundamental Set Operations

The universe (of discourse) *U* is the set of all things

U defines the context we work in to avoid Russell's Paradox

A ∪ B means "A union B", or the set containing all elements that are either in A or in B

- A = {1, 2, 3} and B = {4, 5, 6} means A ∪ B = {1, 2, 3, 4, 5, 6}

A ∩ B means "A intersection B", or the set containing all elements that are both in A and B

- A = {1, 2, 3} and B = {3, 4, 5} means A ∩ B = {3}

A ∩ B = ∅ means "A and B are disjoint", or their intersection is empty

- A = {1, 2, 3} and B = {4, 5, 6} means A ∩ B = ∅ or { }

A − B (or A \ B) means "the difference of A and B", or the set of all elements in A but not in B

- A − B = {x | x ∈ A and x ∉ B}
- Also called "the complement of B with respect to A"

A′ is "the complement of A", or everything in the universe that is not A **...?**

A Δ B = (A ∪ B) ∩ (A ∩ B)′ = (A \ B) ∪ (B \ A)

Venn Diagrams = model of sets

|S| means "the cardinality of S, or the measure of how many elements S has"

- |{1, 2, 3}| = 3
- |∅| = 0
- |{1}, {1{2, 3}}| = 2

The power set P(S) of a set S is the set of all subsets of S

- P(S) = {x | x ⊆ S}
- Sometimes P(S) is written as 2 to the power of S

Identity
Domination
Idempotent
Double complement
Commutative
Associative
De Morgan

12th of February (Sets)

Relations

A list is a structure representing an *ordered* collection of zero or more objects which may include *duplicates*

- Notation: [ ], ( ), ⟨ ⟩

- Elements in a list have positions
- Finite lists have a length

Cartesian product of two sets, a set of pairs (lists of length x 2)

- A x B = { ⟨a, b⟩ : a ∈ A and b ∈ B}
- A x A = A to the power of 2

N-tuples are sequences of lengths n, whose order matters

- (a) is a singles
- (a, b) is a pair
- (a, b, c) is a triple

Relations are sets of n-tuples

A relation in the set A is a binary relation of type A to the power of 2

A binary relation is a relation of type A x B or A to the power of 2

- FollowsCourse = {⟨x, y⟩ : x follows y} ⊆ Students x Courses
  - Alternative infix notation: x FollowsCourse y instead of ⟨x, y⟩ ∈ FollowsCourse

---

Graphs

A graph G is generalisation of the simple concept of a set of dots, links, edges, or arcs

- G = (V, E), where V is the set or vertices (singular: vertex), and E is the set of paired vertices or edges

Undirected graphs

- A non-empty set of vertices V, and a set of unordered, distinct edges E
  - G(V, E) V = {u, v, w} and E = {{u, v}, {v, w}, {u, w}}

Directed graphs

- A non-empty set of vertices V, and a set of directed edges E, consisting of ordered pairs
  - G(V, E) V = {u, v, w} and E = {(u, v), (v, w), (u, w)}

Definition: A graph is Eulerian if it is both connected and has a closed trail (a walk with no repeated edges) containing all edges of the graph

Theorem: A graph is Eulerian if every node in the graph has an even degree

The adjacency matrix of a graph G is the matrix A defined as adjacency between nodes u and v

- if the set of edges E includes (u, v), then A[u, v] has a value of 1
- if there is no direct path to u to v, then A[u, v] has a value of 0

Trees

A tree is a connected, undirected graph with no simple circuits

- A circuit (cycle) is a path of length ≥ 1 that begins and ends at the same vertex
- Ex. tournaments and family trees

Once a tree has a designed root node, it is possible to assign a direction to each edge

- Root node is the one at the top
- Internal nodes are the nodes in between
- Parent relations, one above the other
- Sibling relations, the same order or the same parent
- Leaf node is the last node

A rooted tree is an n-ary tree if every internal vertex has no more than n children

- A tree is full n-ary if every internal vertex has exactly n children
- An n-ary tree with n=2 is called a binary tree

The level or depth of a vertex v in a rooted tree is the length of the unique path from the root to this vertex

The height (also called depths) of a rooted tree is the maximum of the levels of vertices

18th of February (Logic)

DNF (Disjunctive Normal Form)

A disjunctive normal form is a disjunction where the formulas are conjunctions of literals

A literal is a propositional variable or the negation of a propositional variable

Make a formula from the Trues in a truth table

CNF (Conjunctive Normal Form)

An AND or OR propositional logic formula, it is the standard form used in SAT solvers and DPLL algorithms. Transform any propositional formula into CNF:

Eliminate Equivalences

$(A \leftrightarrow B)$ is equivalent to $(A \rightarrow B) \land (B \rightarrow A)$

$(A \rightarrow B)$ is equivalent to $(\neg A \lor B)$

Move NOTs inward (De Morgan's law)

$\neg (A \land B)$ is equivalent to $\neg A \lor \neg B$ → Not and is or

$\neg (A \lor B)$ is equivalent to $\neg A \land \neg B$ → Not or is and

$A \lor B$ is equivalent to $\neg (\neg A \land \neg B)$ → When either A or B is true, they can't both be false

Distributive law (distribute OR over AND)

$A \lor (B \land C)$ into $(A \lor B) \land (A \lor C)$

Sheffer Stroke (Nand-operation)

$\Phi \mid \psi$ → not both Φ and ψ

- $\Phi \mid \psi \equiv \neg (\Phi \land \psi)$
- $\Phi \mid \Phi = \neg \Phi$

DPLL

An algorithm for deciding satisfiability of propositional formulas.

It has:

A literal

- From proposition A you get two literal
  - ex. A
  - ex. ¬A
- Pure literal
  - literal that occurs in only one clause

A variable is one of the simple statements (the letters) that make up the input statements

- The name of a statement, it doesn't include negation

A clause is a disjunction of literals

- Clauses are the building blocks of CNF
- Each parenthesized group is a clause most of the time
- Single-literal clause (unit clause)
  - ex. P
- Multi-literal clause
  - ex. (¬P ∨ Q)

The polarity of a literal indicates whether its positive or negative

- Positive polarity → not negated
  - ex. A, B, C
- Negative polarity → negated
  - ex. ¬A, ¬B

The easy cases of DPLL are:

- If a clause contains ¬A ∨ A it can be removed, because the case will be satisfied in any case
- If there is a pure literal then that literal can be set to true, because this choice doesn't affect any other clauses
- If there is a unit clause that literal can be set to true, because this is the only way the clause can be satisfied (ex. if the literal ¬A is set to true, it will become false)

Rules of DPLL:

- Unit propagation: the literal in a unit clause must be true, otherwise the clause can't be satisfied
- Pure literal elimination: set the literal so that the polarity is true (ex. False for ¬A and True for A)
- Branching (decision): If nothing forced remains pick a variable and try it out with either True or False, recursively run DPLL. If that branch returns unsatisfied backtrack and try the other one.
- If all clauses become satisfied → SAT and if an empty clause appears → UNSAT

Example 1:

(A ∨ B) ∧ (¬A ∨ C) ∧ (¬B ∨ C) ∧ (¬C ∨ D)

Clauses:

- C1: A ∨ B
- C2: ¬A ∨ C
- C3: ¬B ∨ C
- C4: ¬C ∨ D

Step 1: Check Unit Clauses

- None (no clause is the size of 1 literal)

Step 2: Check Pure Literals

- A appears only as A in C1 and as ¬A in C2 — both polarities → not pure.
- B appears B in C1 and ¬B in C3 → not pure.
- C appears only as C in C2 and C3 and only as ¬C in C4 → both polarities → not pure.
- D appears only as D in C4 and never as ¬D → D is a pure literal (positive).

Which means D = True, so you can drop C4

Step 3: Branch (Decision)

Try A = True

- C1: (A ∨ B) is satisfied (drop it).
- C2: (¬A ∨ C) → (False ∨ C) → becomes unit (C) → so C = True.
- C3: (¬B ∨ C) → with C = True becomes satisfied (drop it).

No conflict → SAT found

* B is unconstrained (not forced by unit clauses or needed to satisfy any remaining clause), so you can assign it True or False, whatever you want, it won't make a difference in the satisfiability.

Example 2:

(A ∨ B) ∧ (¬A ∨ C) ∧ (¬B ∨ C) ∧ (¬C ∨ D) ∧ (¬D)

Clauses:

- C1: A ∨ B
- C2: ¬A ∨ C
- C3: ¬B ∨ C
- C4: ¬C ∨ D
- C5: ¬D

Step 1: Check Unit Clauses

- Yes → C5 = ¬D
  - Unit propagation: set D to False
  - C4: (¬C ∨ D) becomes (¬C ∨ False) → **(¬C)** — now a unit clause.
  - Others unchanged.
- Yes → C4 = ¬C
  - Unit propagation: set C to False
  - C2: (¬A ∨ C) → (¬A ∨ False) → (¬A) → unit → set A = False.
  - C3: (¬B ∨ C) → (¬B ∨ False) → (¬B) → unit → set B = False.
  - C1: (A ∨ B) → (False ∨ False) → empty clause (contradiction)

Empty clause → UNSAT

*The other steps weren't needed for this example

SAT Solving and Entailment

A sentence is satisfiable if it is true in some interpretation, there exists a model

A sentence is unsatisfiable if it is true in no interpretation, there is no model

A sentence is valid if it is true in all interpretations, also called a tautology

A sentence is entailed by a KB, i honestly don't know what I'm seeing ???

Satisfiability Solving

It is the question: "Is there a model?", the task of finding one or more models of the KB

It is an old but still active field of research, used in hardware/software verification, planning & scheduling, control, protocol design, etc.

Entailment as Satisfiability

We can check what follows from a knowledge base by checking unsatisfiability, or nonexistence of a model